

Binary Search Tree

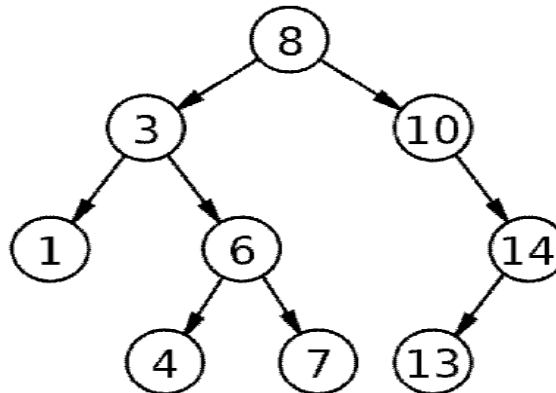
CSE 2215 - Lecture 11 - Summer 2023

Instructor : Fahmid Al Rifat, Lecturer, Dept. of CSE , UIU

fahmid@cse.uiu.ac.bd

Binary Search Tree

- A **binary search tree (BST)** is a node-based binary tree data structure which has the following properties:
 - The left subtree of a node contains only nodes with keys less than the node's key.
 - The right subtree of a node contains only nodes with keys greater than or equal to the node's key.
 - Both the left and right subtrees must also be binary search trees.

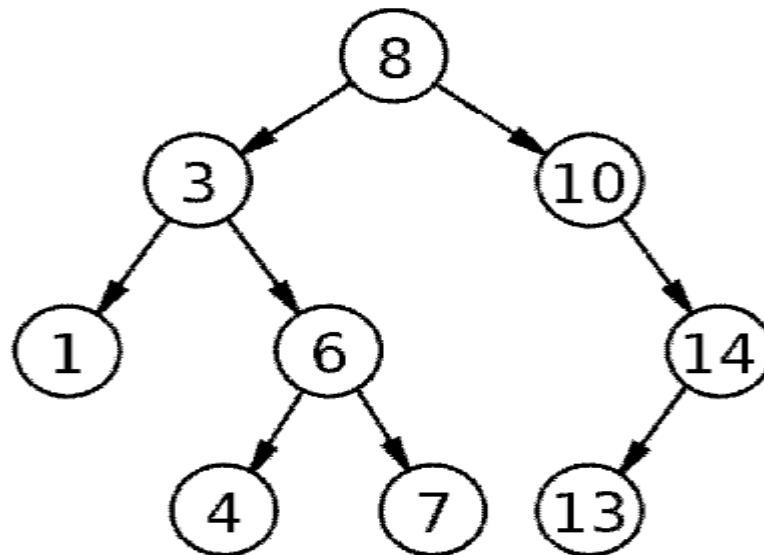


Binary Search Tree

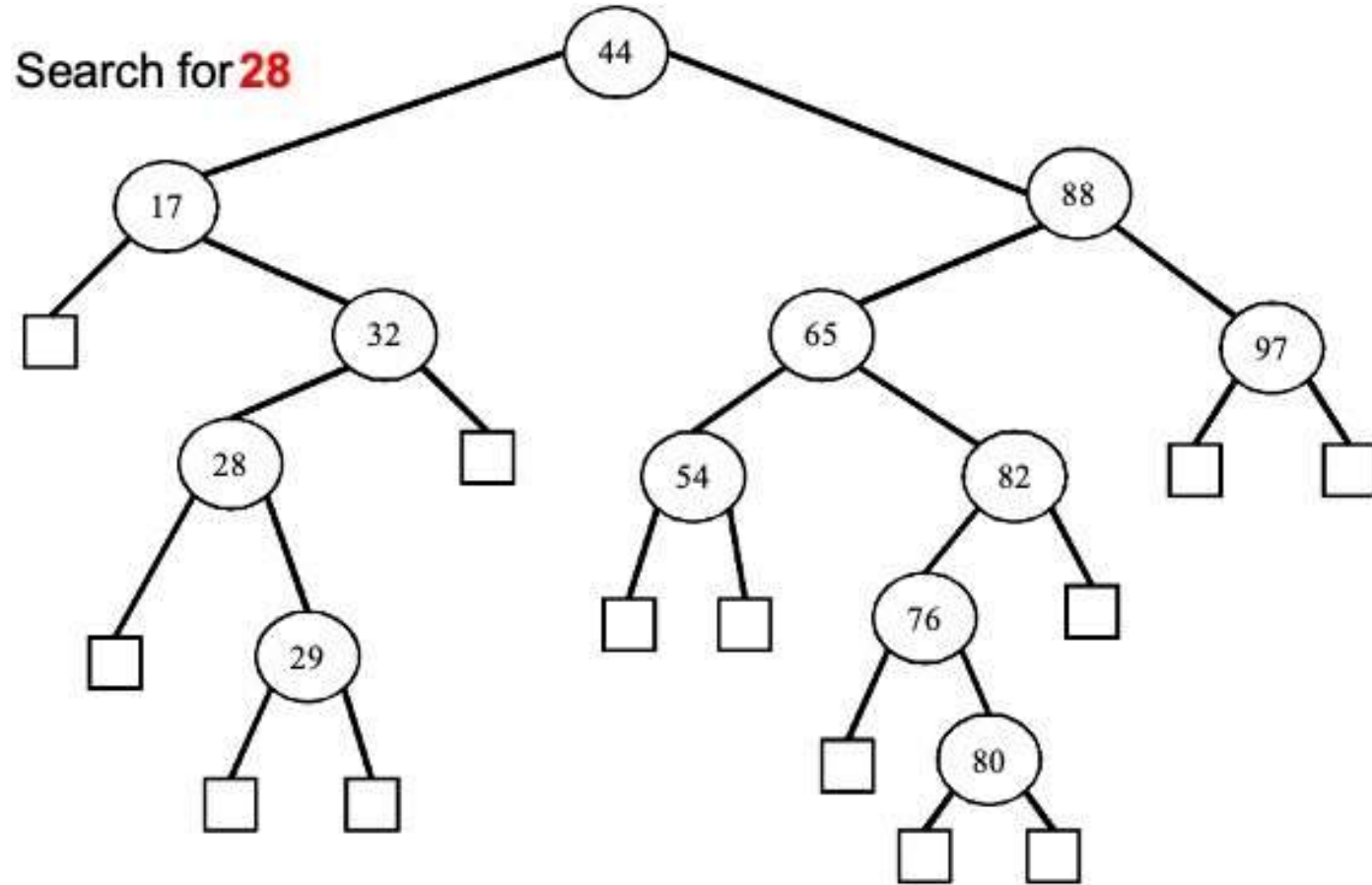
- Firstly, it is a binary tree
- It is represented by a linked data structure
- It combines
 - the advantage of an array -- the ability to do a binary search with
 - the advantage of a linked list -- its dynamic size
- The efficiency of all of the operations is $O(h) = O(\log n)$, only if the tree is reasonably height-balanced
- Each node contains at least the following fields
 - *key*: an identifying field
 - *left*: pointer to a left child (may be NIL)
 - *right*: pointer to a right child (may be NIL)
 - *p*: pointer to the parent node (NIL for root)

Binary Search Tree

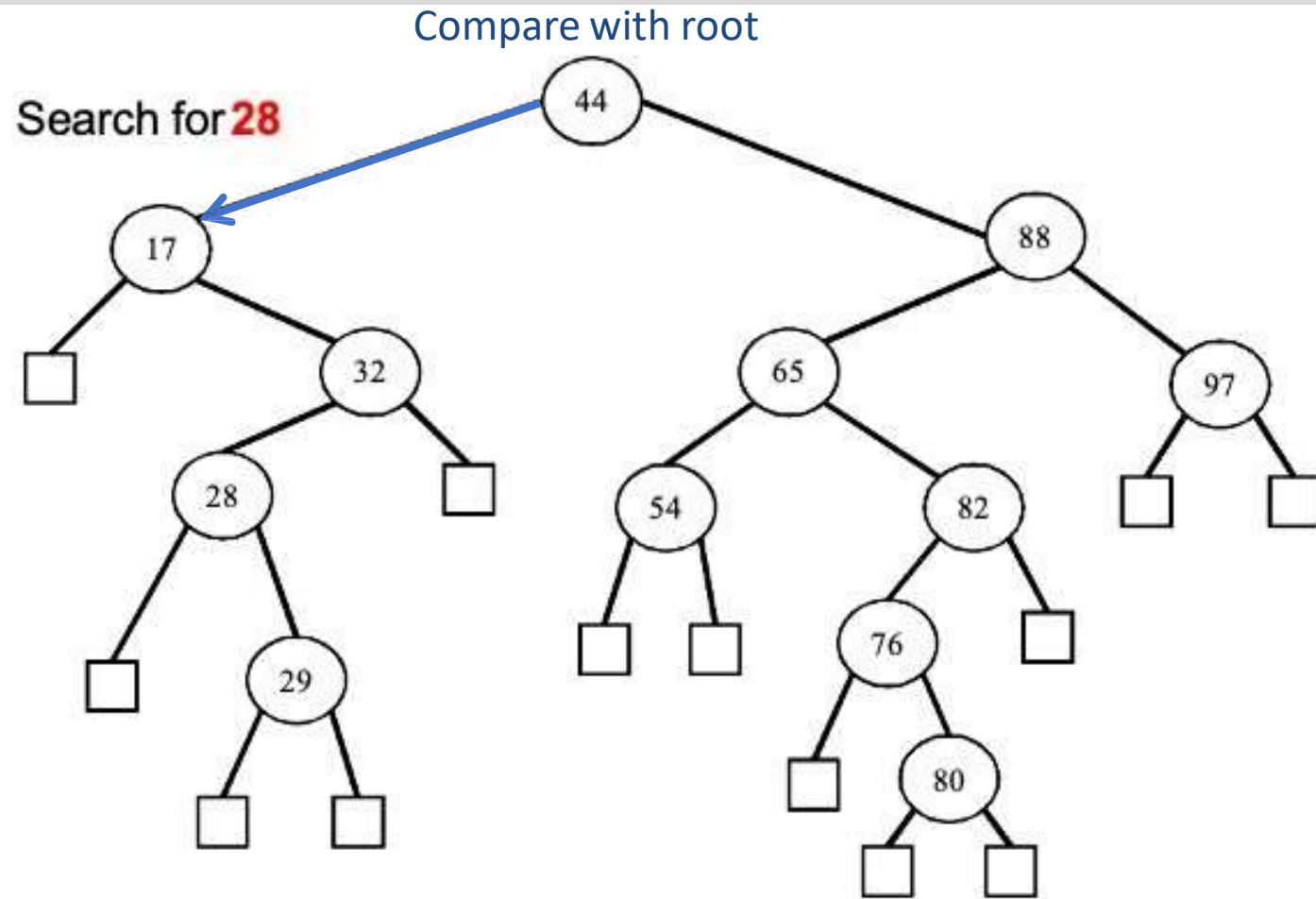
- Secondly, the binary-search-tree property
 - For any node x ,
 - $\text{key}[y] < \text{key}[x]$ if y in left subtree of x
 - $\text{key}[y] \geq \text{key}[x]$ if y in right subtree of x



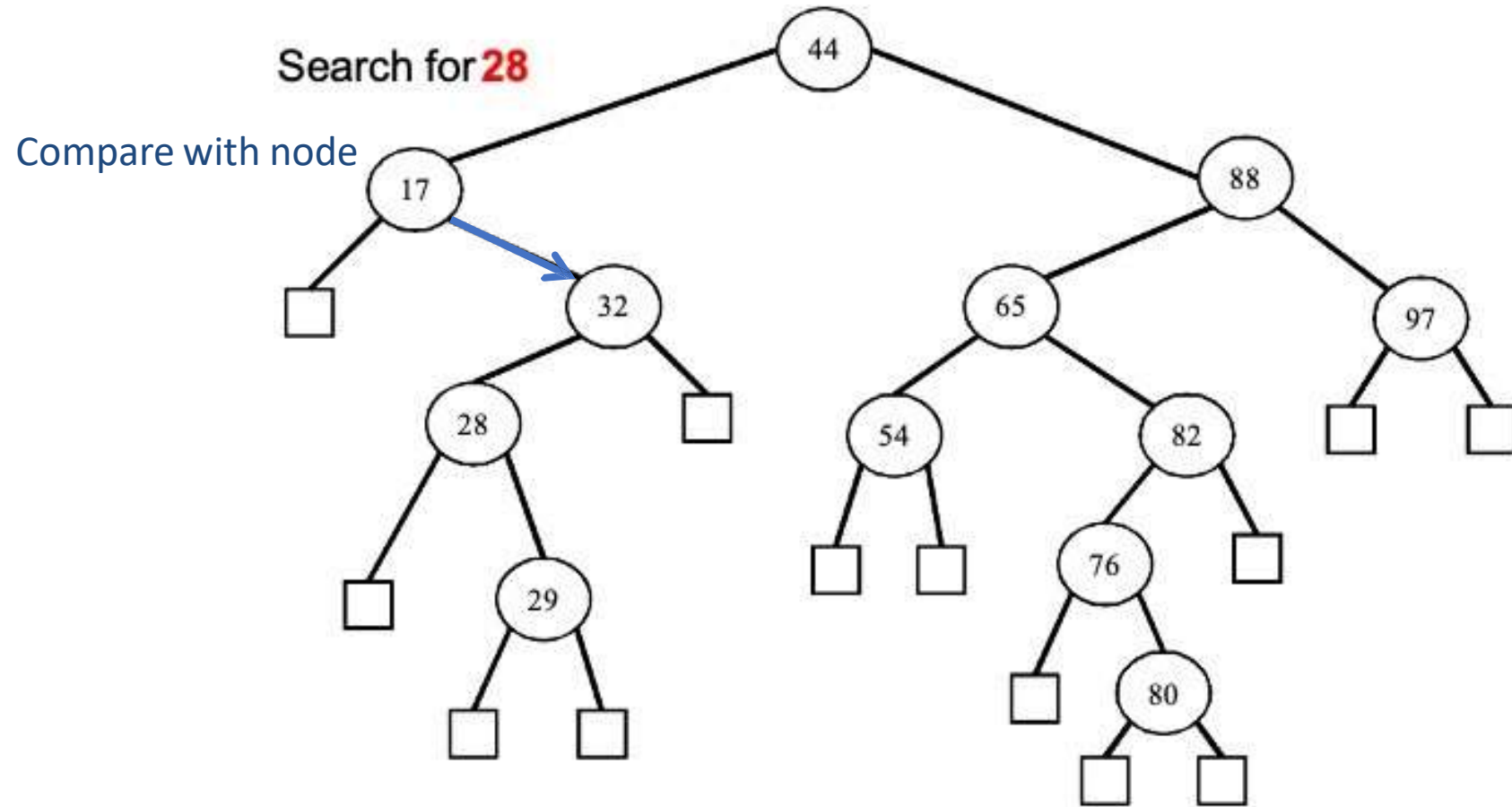
Operation on BST: Search



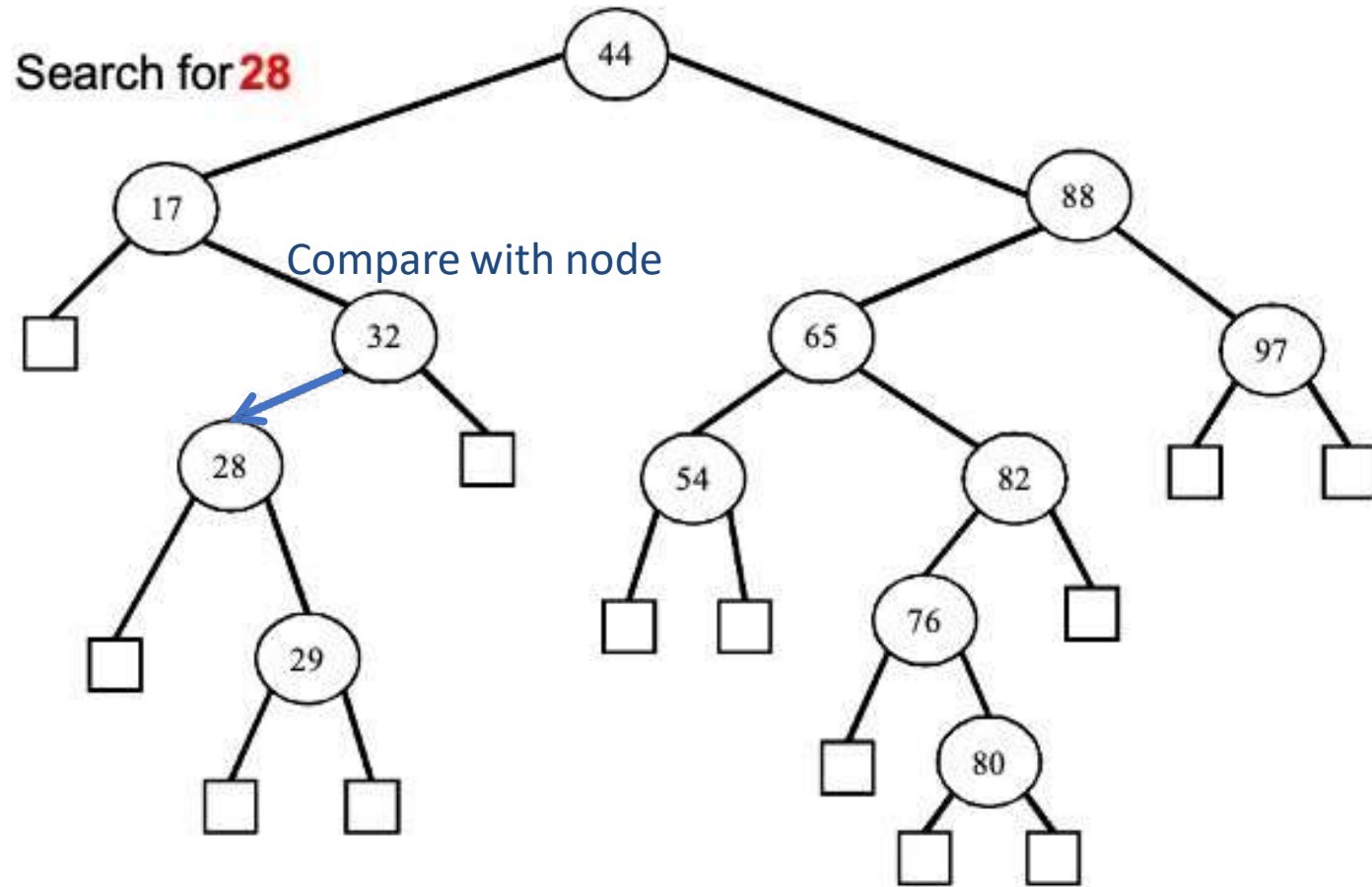
Operation on BST: Search



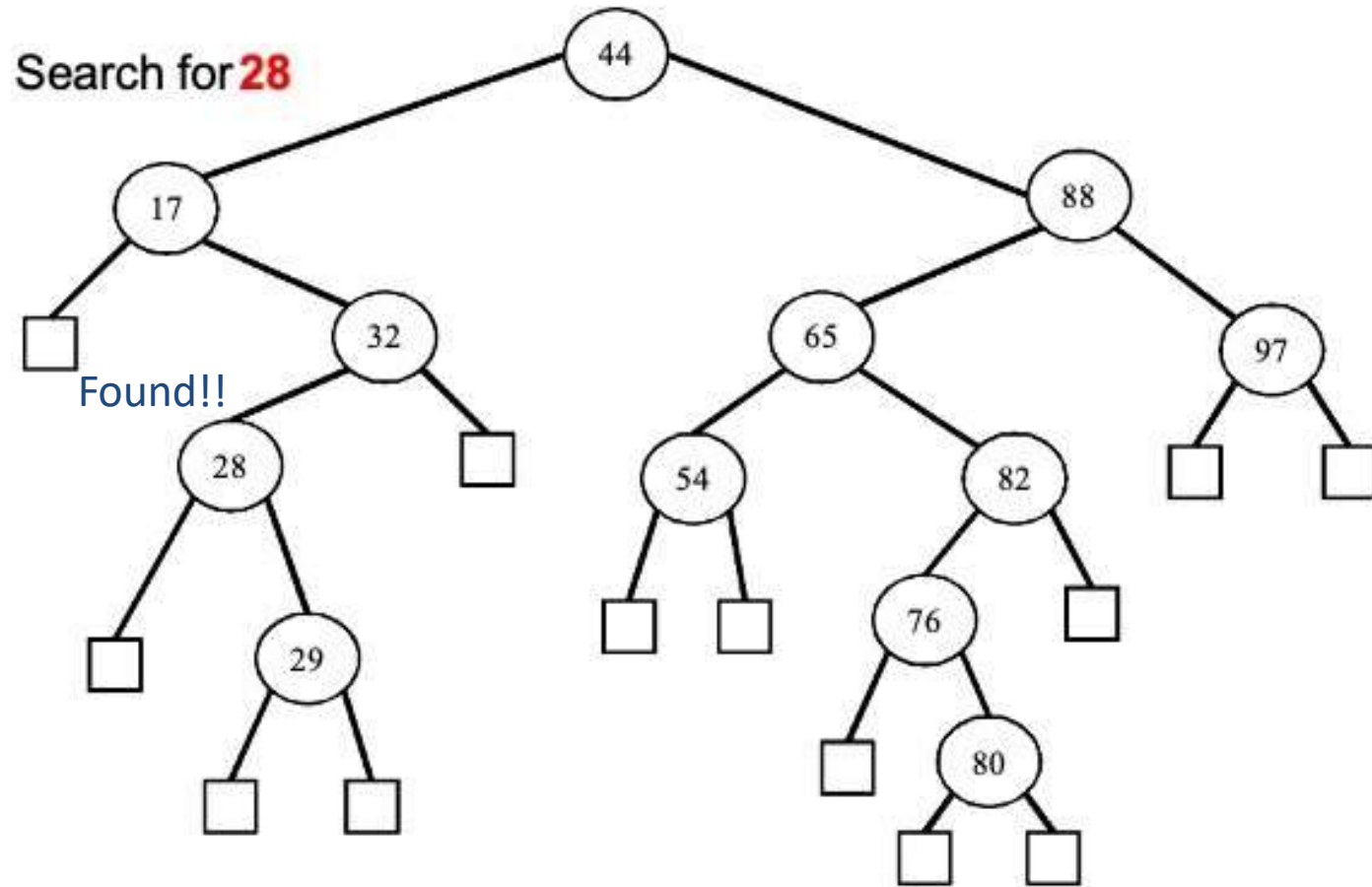
Operation on BST: Search



Operation on BST: Search



Operation on BST: Search



Operation on BST: Search

- Given a key k and a pointer to the root, Tree-Search returns a pointer to a node with key k or NIL:

```
Tree-Search(x, k)
    //return if found
    if(x == NIL or k==key[x])
        return x;
    //else recursively search in subtree
    if(k < key[x])
        return Tree-Search(left[x], k);
    else return Tree-Search(right[x], k);
```

Operation on BST: Search

- Here is another function that does the same:

```
Tree-Search(x, k)
    while (x != NIL and k != key[x]){
        if (k < x->value)
            x = x->left;
        else
            x = x->right;
    }
    return x;
```

Operation on BST: Min & Max

TREE-MINIMUM(x)

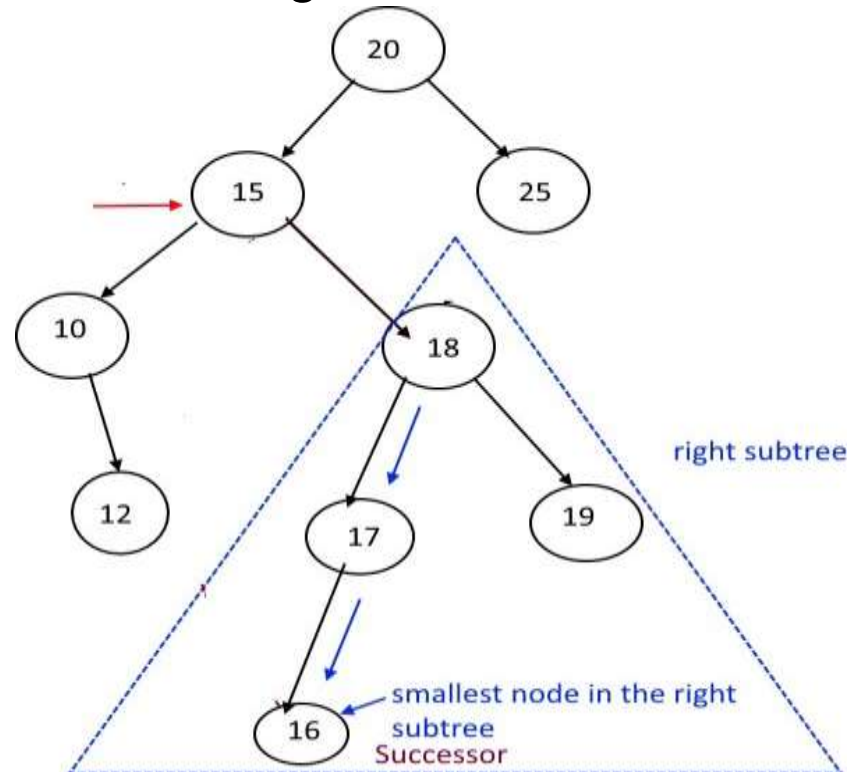
```
1  while  $left[x] \neq \text{NIL}$   
2      do  $x \leftarrow left[x]$   
3  return  $x$ 
```

TREE-MAXIMUM(x)

```
1  while  $right[x] \neq \text{NIL}$   
2      do  $x \leftarrow right[x]$   
3  return  $x$ 
```

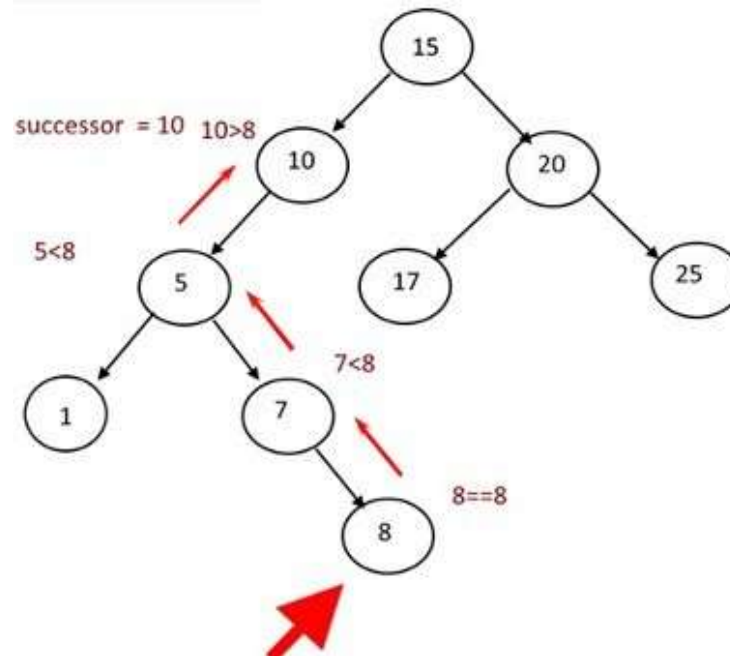
Operation on BST: Successor

- Two cases:
 - Case 1: x has a right subtree:
 - ◆ successor is minimum node in right subtree



Operation on BST: Successor

- Two cases:
 - Case 2: x has no right subtree:
 - ◆ successor is first ancestor of x whose left child is also ancestor of x
 - ◆ Intuition: As long as you move to the left up the tree, you're visiting smaller nodes.



Operation on BST: Successor (Code)

TREE-SUCCESSOR(x)

```
1  if  $right[x] \neq \text{NIL}$ 
2      then return TREE-MINIMUM( $right[x]$ )
3   $y \leftarrow p[x]$ 
4  while  $y \neq \text{NIL}$  and  $x = right[y]$ 
5      do  $x \leftarrow y$ 
6       $y \leftarrow p[y]$ 
7  return  $y$ 
```

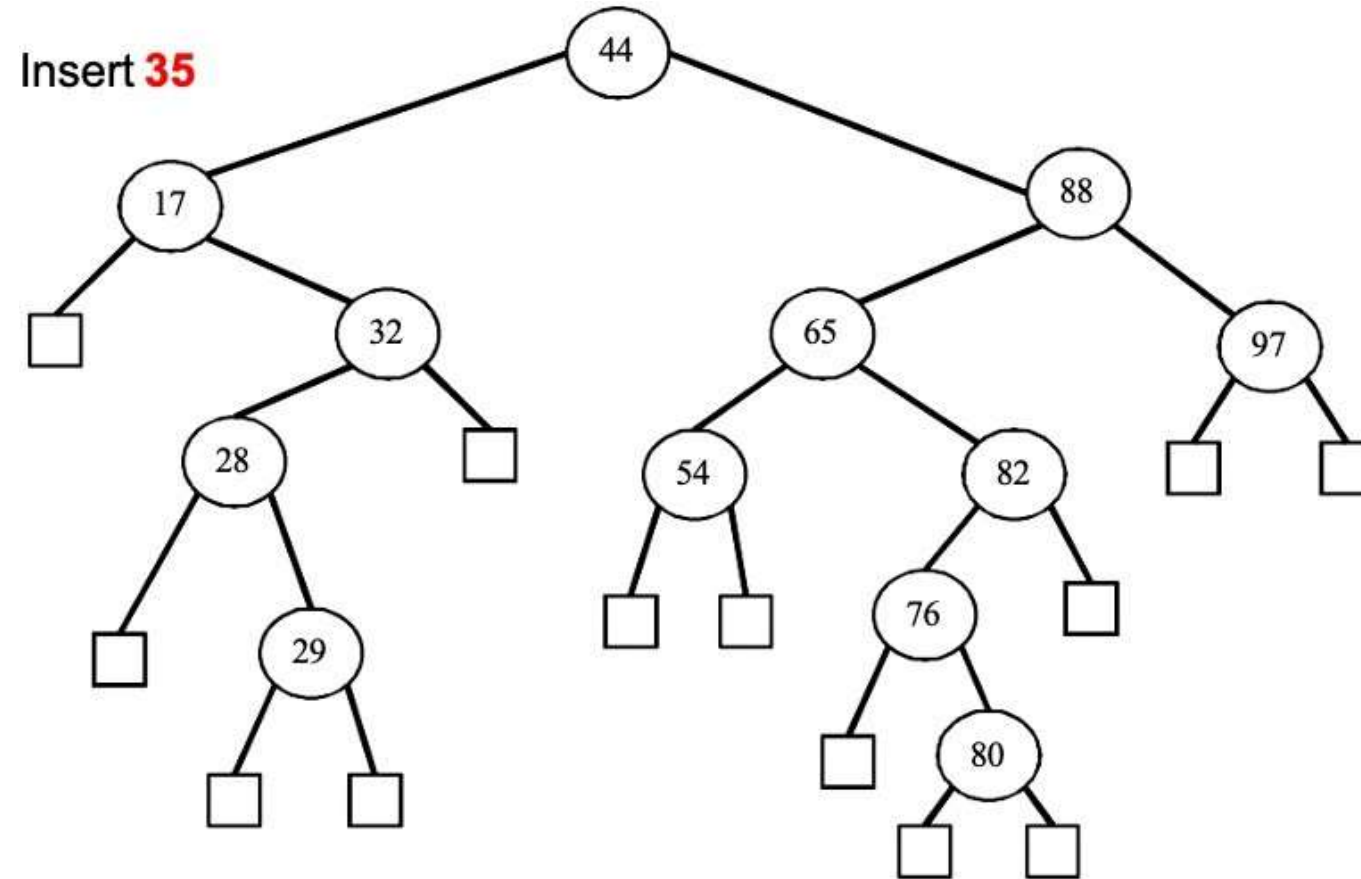
Operation on BST: Predecessor

- Two cases:
 - similar to Successor algorithm

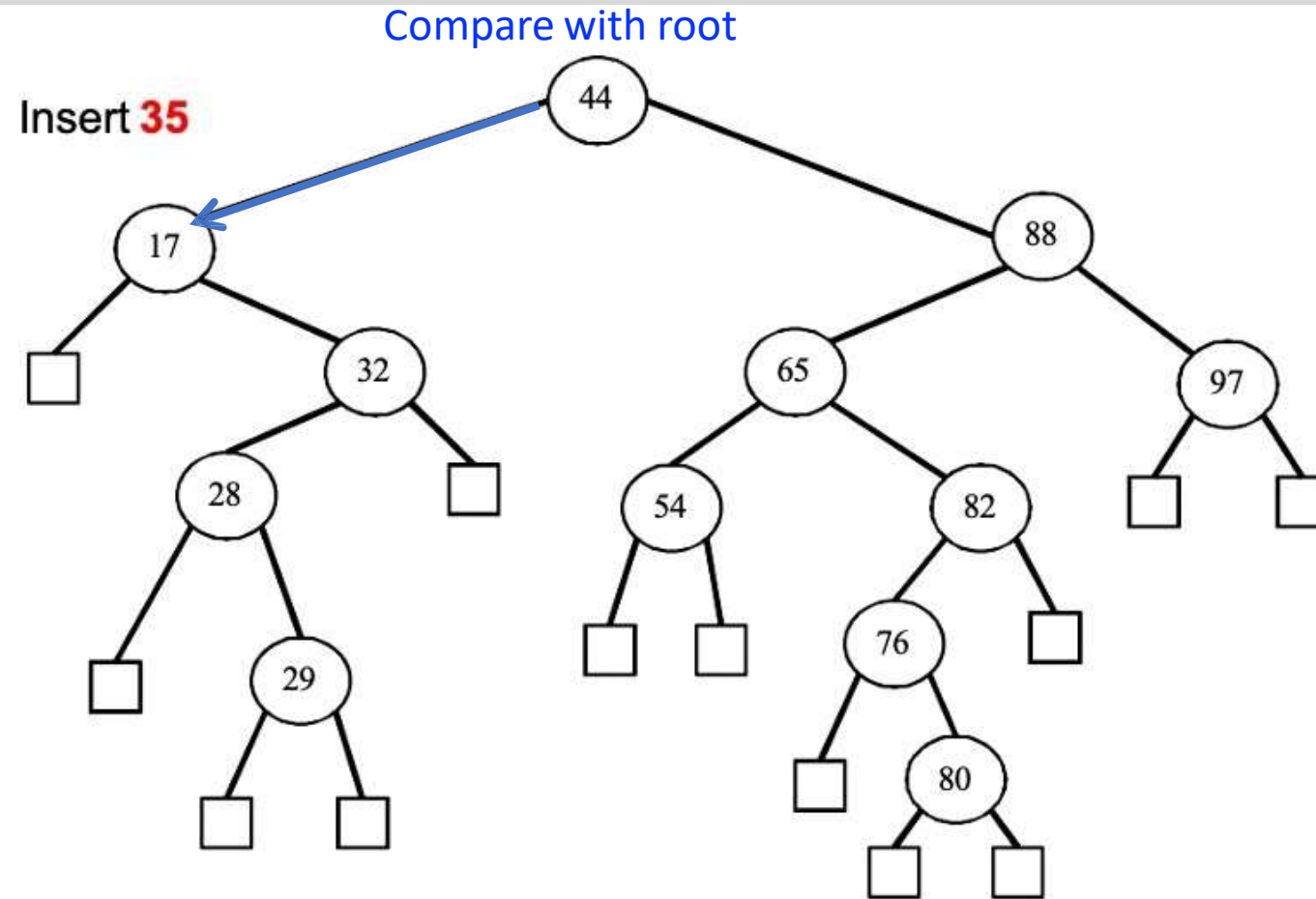
Operation on BST: Insert

- Inserts an element z to the tree so that the binary search tree property continues to hold
- The basic algorithm
 - Like the search procedure as discussed before
 - Insert z in place of NIL
 - Use a “trailing pointer” to keep track of where you came from (like inserting into singly linked list)

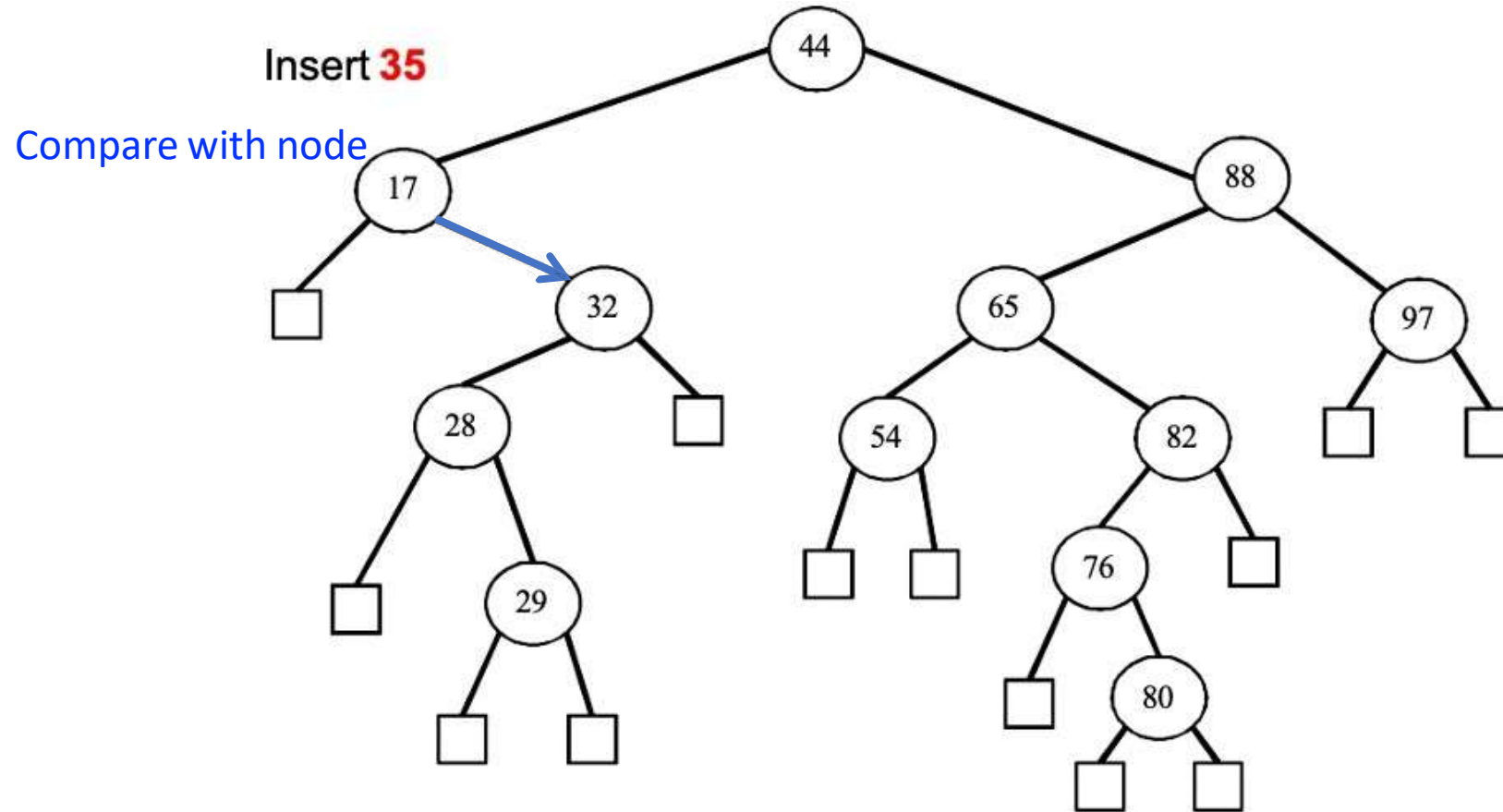
Operation on BST: Insert



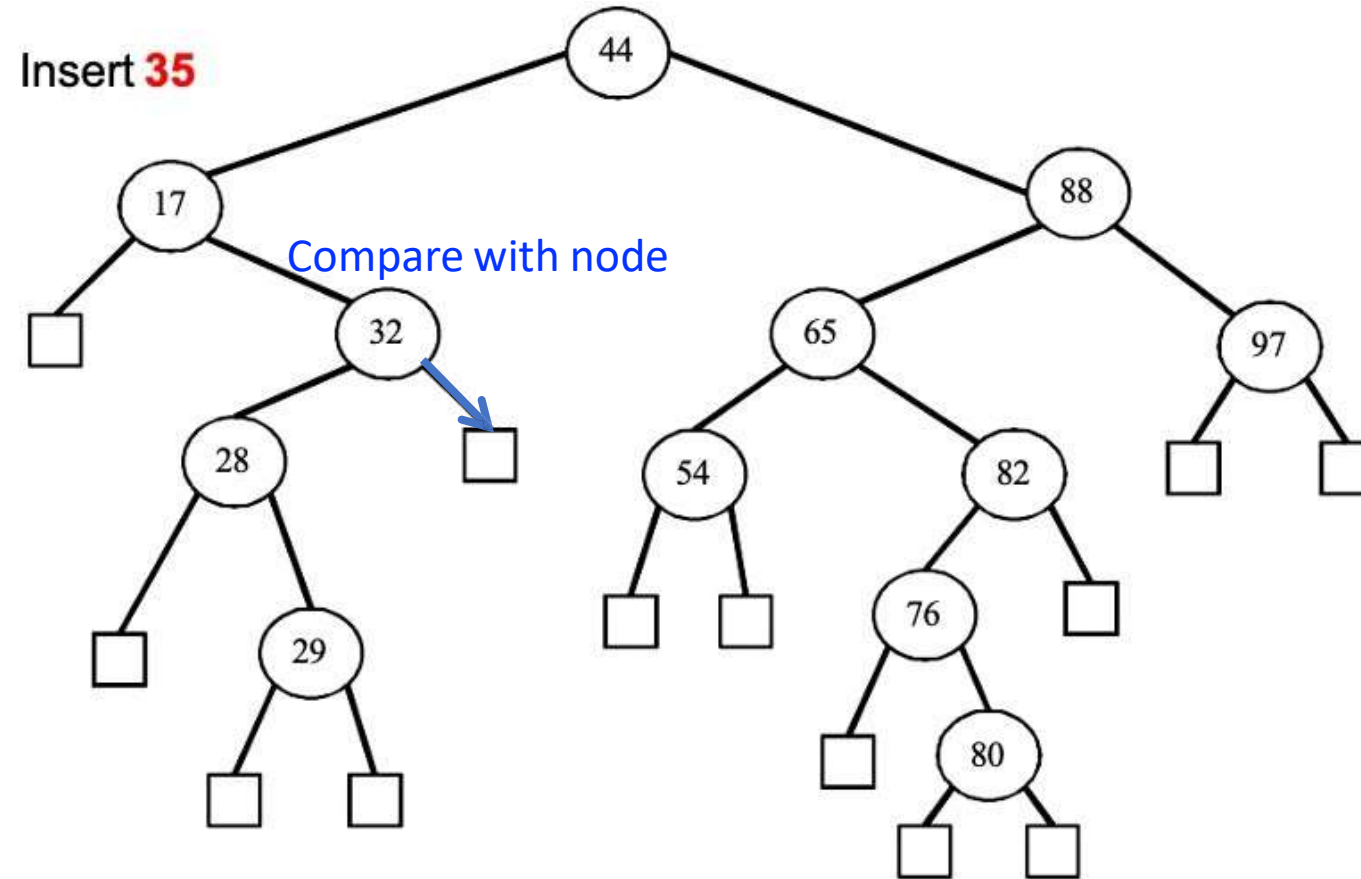
Operation on BST: Insert



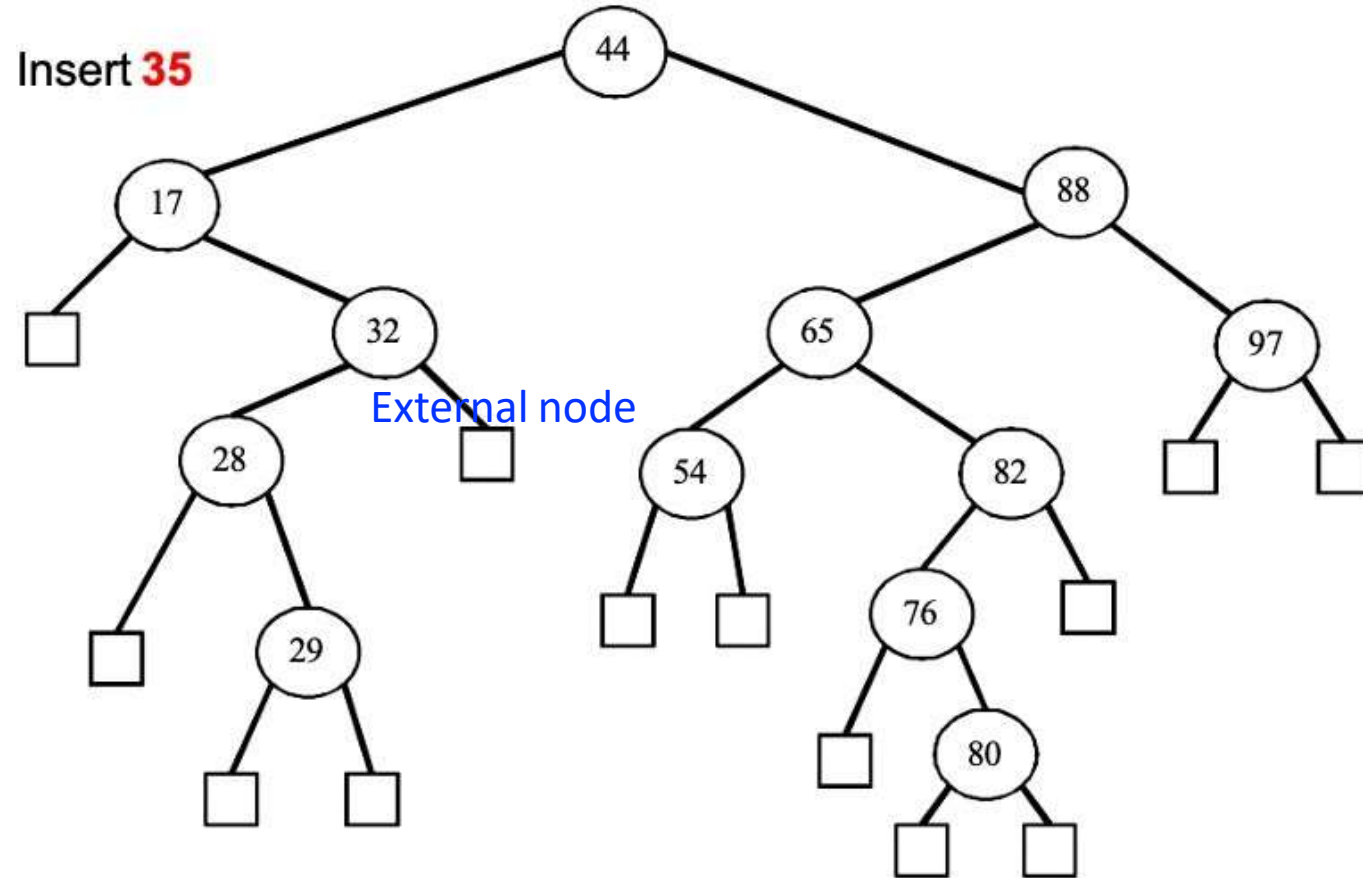
Operation on BST: Insert



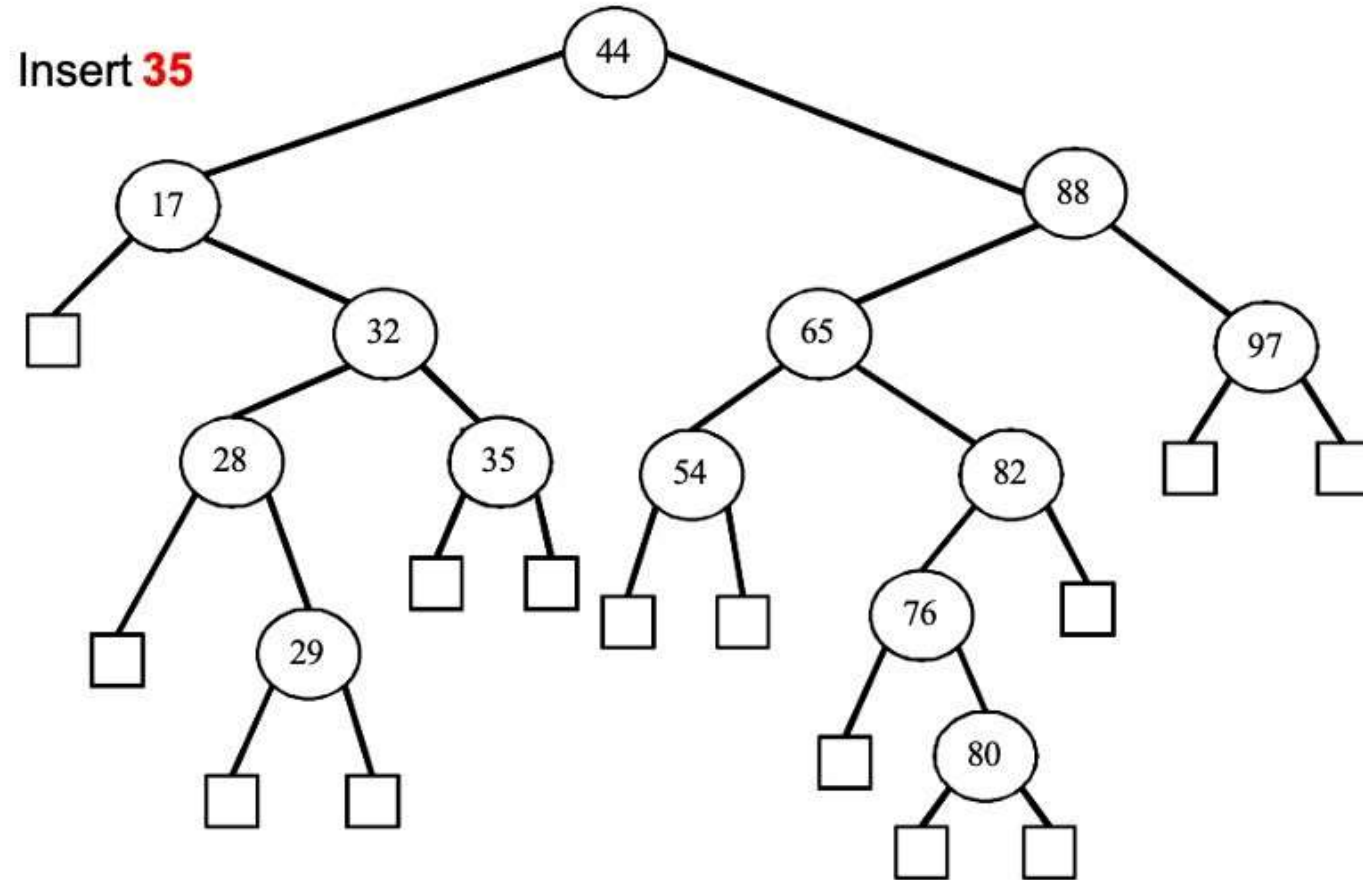
Operation on BST: Insert



Operation on BST: Insert

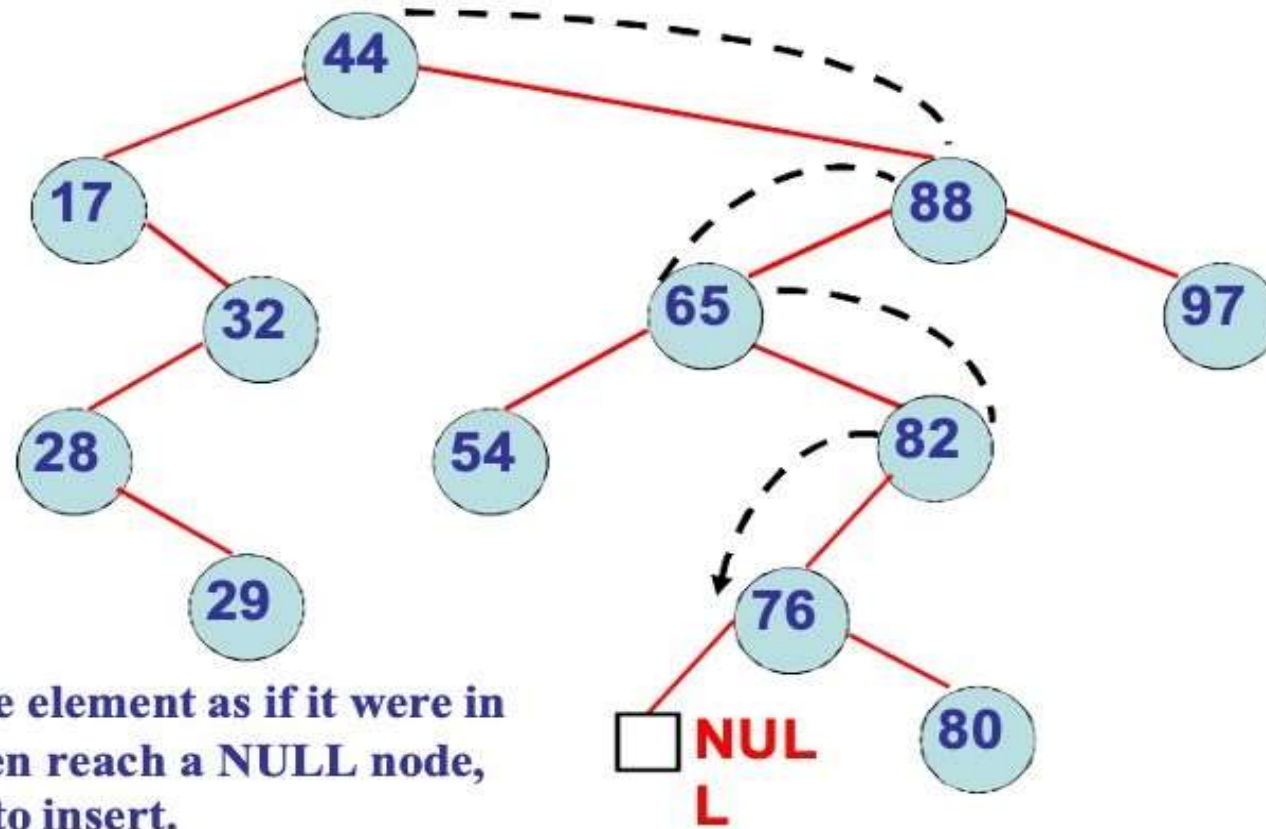


Operation on BST: Insert



Operation on BST: Insert

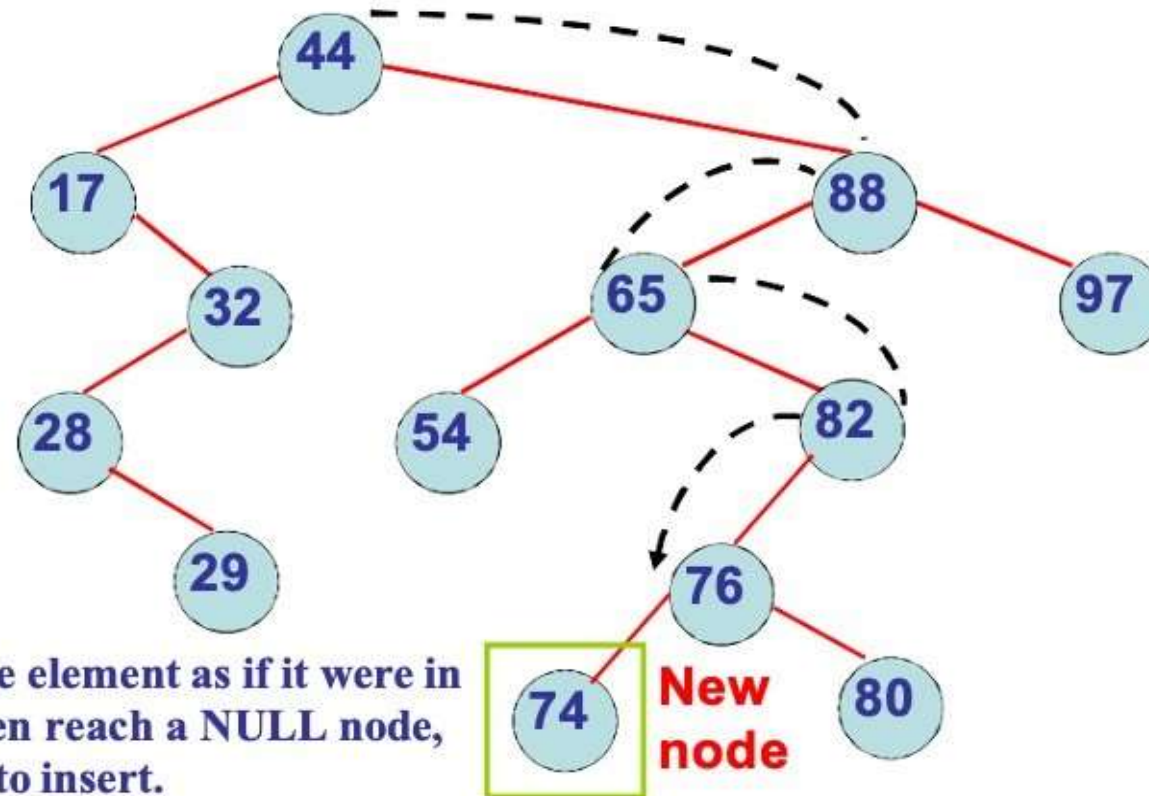
- Example: Insert 74



Search for the element as if it were in the tree. When reach a NULL node, that's where to insert.

Operation on BST: Insert

- Example: Insert 74



Operation on BST: Insert

TREE-INSERT(T, z)

```
1   $y \leftarrow \text{NIL}$ 
2   $x \leftarrow \text{root}[T]$ 
3  while  $x \neq \text{NIL}$ 
4      do  $y \leftarrow x$ 
5          if  $\text{key}[z] < \text{key}[x]$ 
6              then  $x \leftarrow \text{left}[x]$ 
7              else  $x \leftarrow \text{right}[x]$ 
8   $p[z] \leftarrow y$ 
9  if  $y = \text{NIL}$ 
10     then  $\text{root}[T] \leftarrow z$ 
11     else if  $\text{key}[z] < \text{key}[y]$ 
12         then  $\text{left}[y] \leftarrow z$ 
13         else  $\text{right}[y] \leftarrow z$ 
```

▷ Tree T was empty

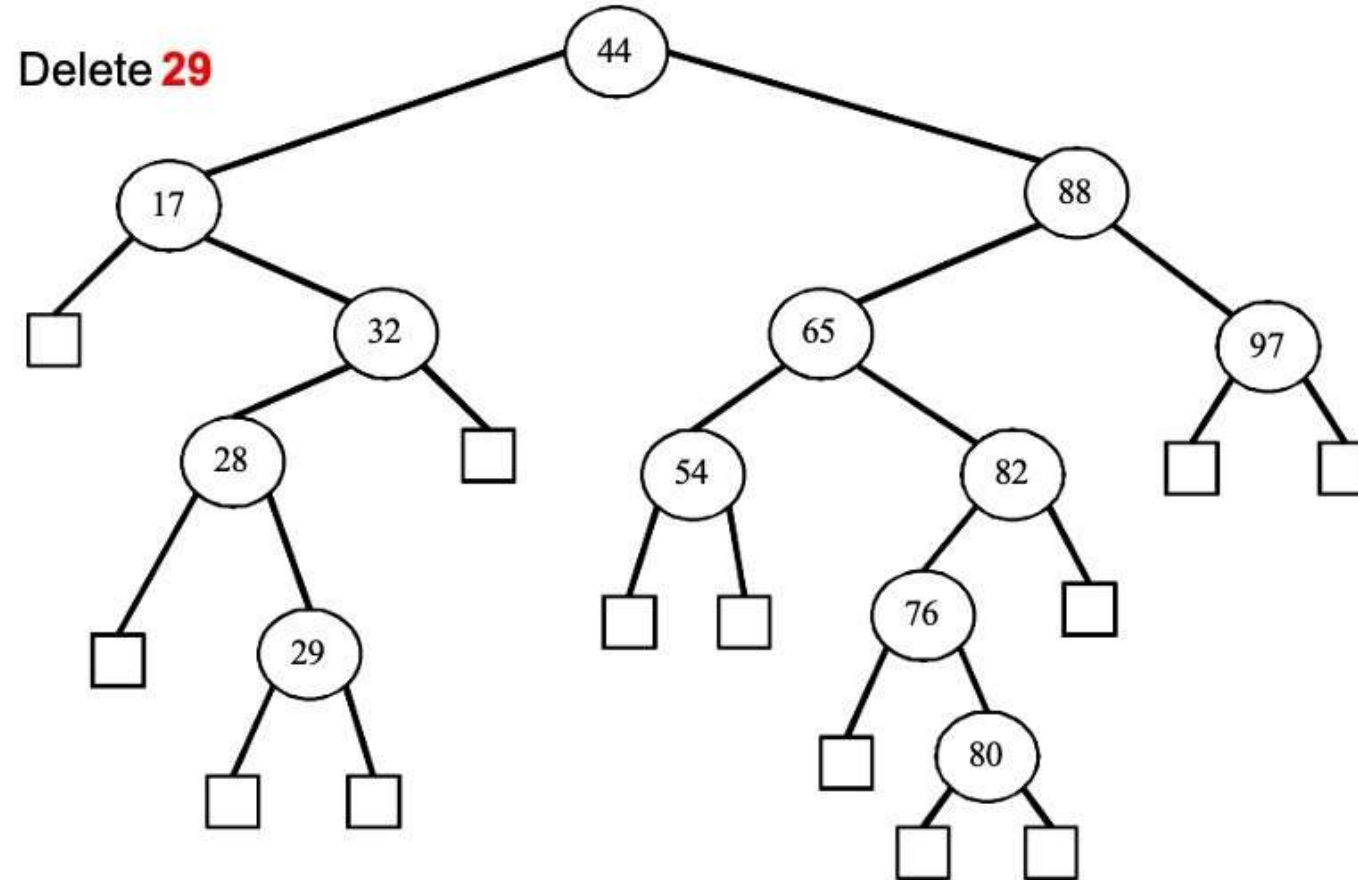
Operation on BST: Insert

- Draw a Binary Search Tree (showing each step) after inserting the following keys:
 - 22, 15, 25, 10, 35, 46, 37, 8, 50

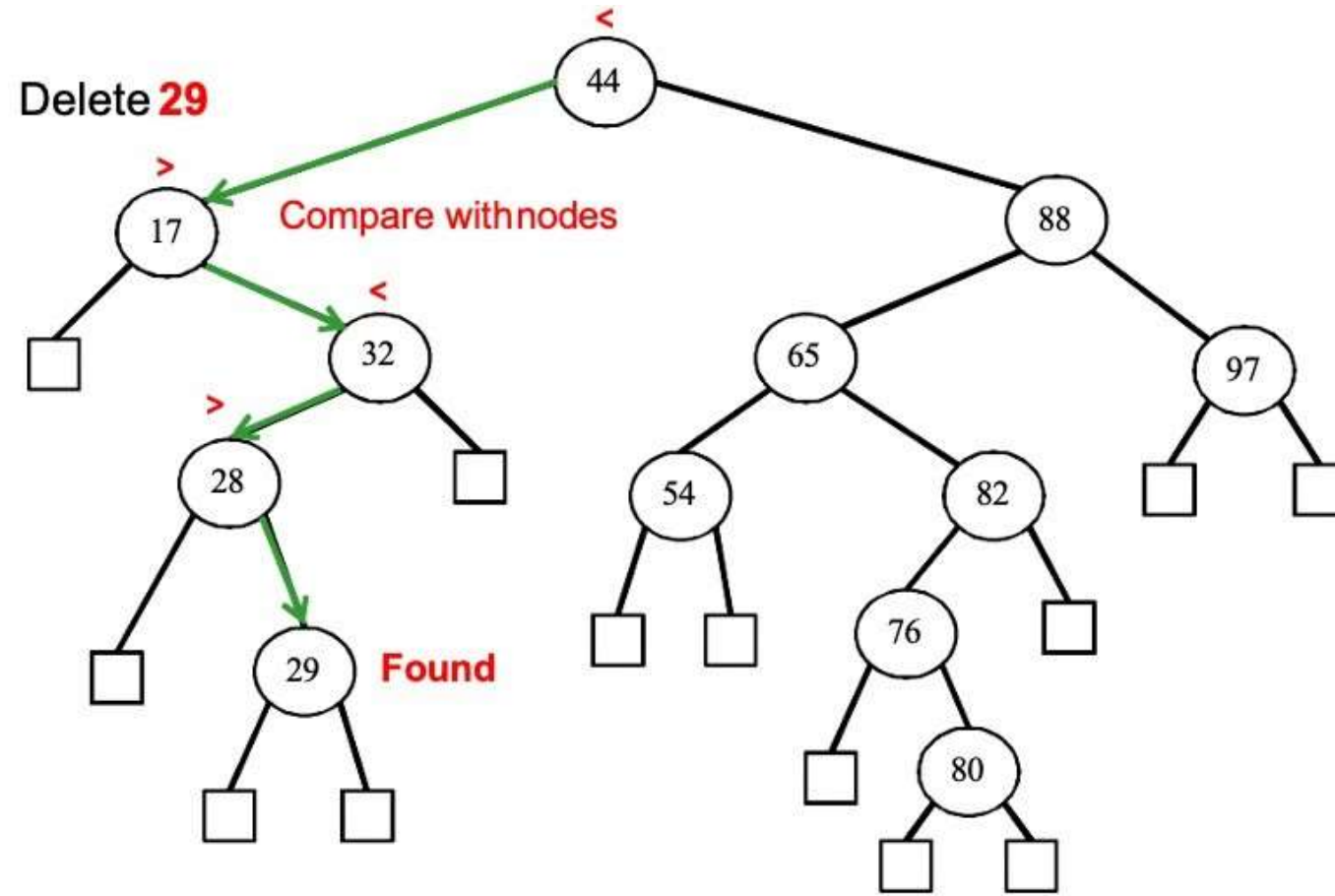
Operation on BST: Delete

- Delete node x
 - 3 cases:
 - x has no children:
 - Remove x
 - x has one child:
 - Splice out x
 - x has two children:
 - Find its inorder successor y
 - Replace x with y
 - Delete y

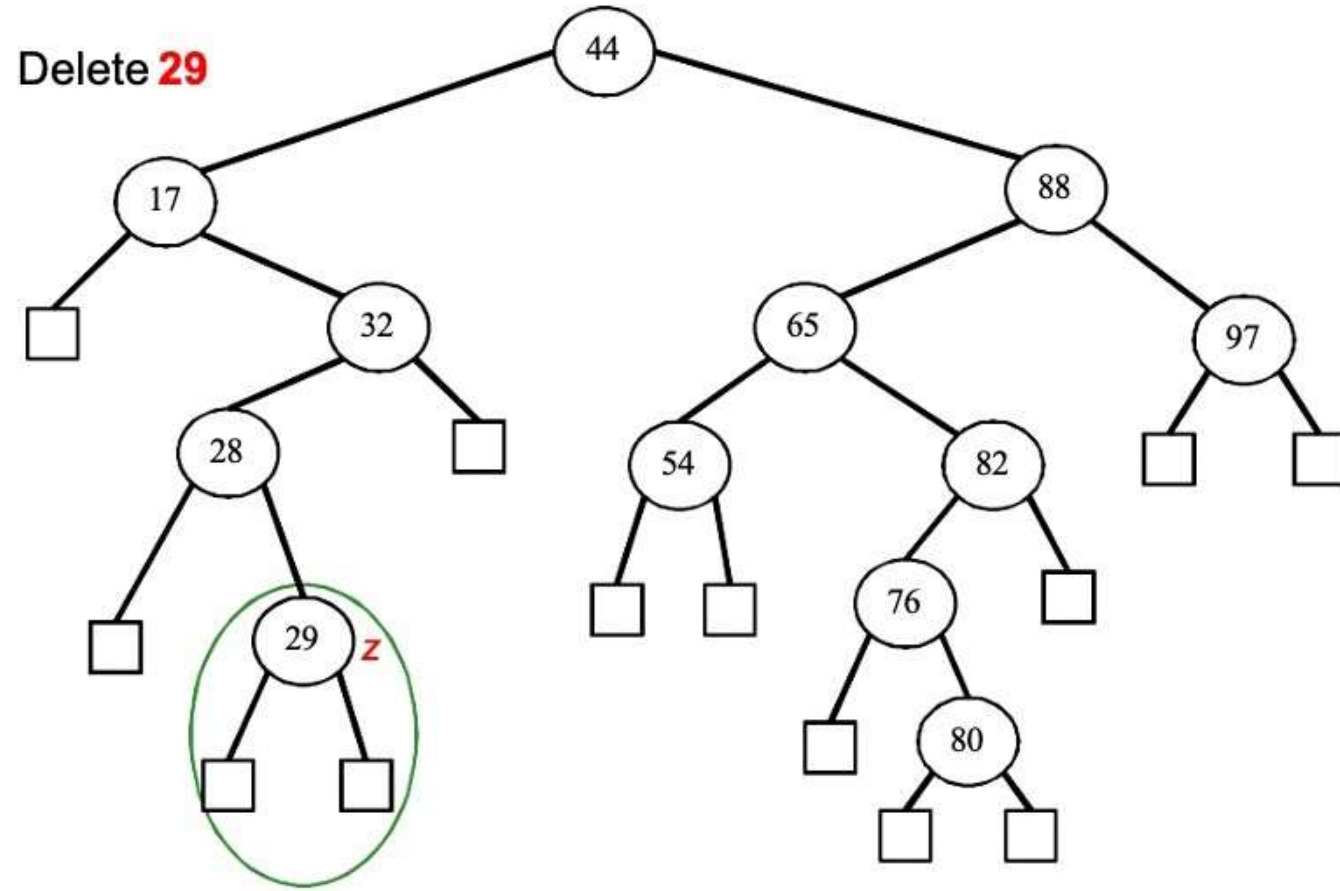
Operation on BST: Delete



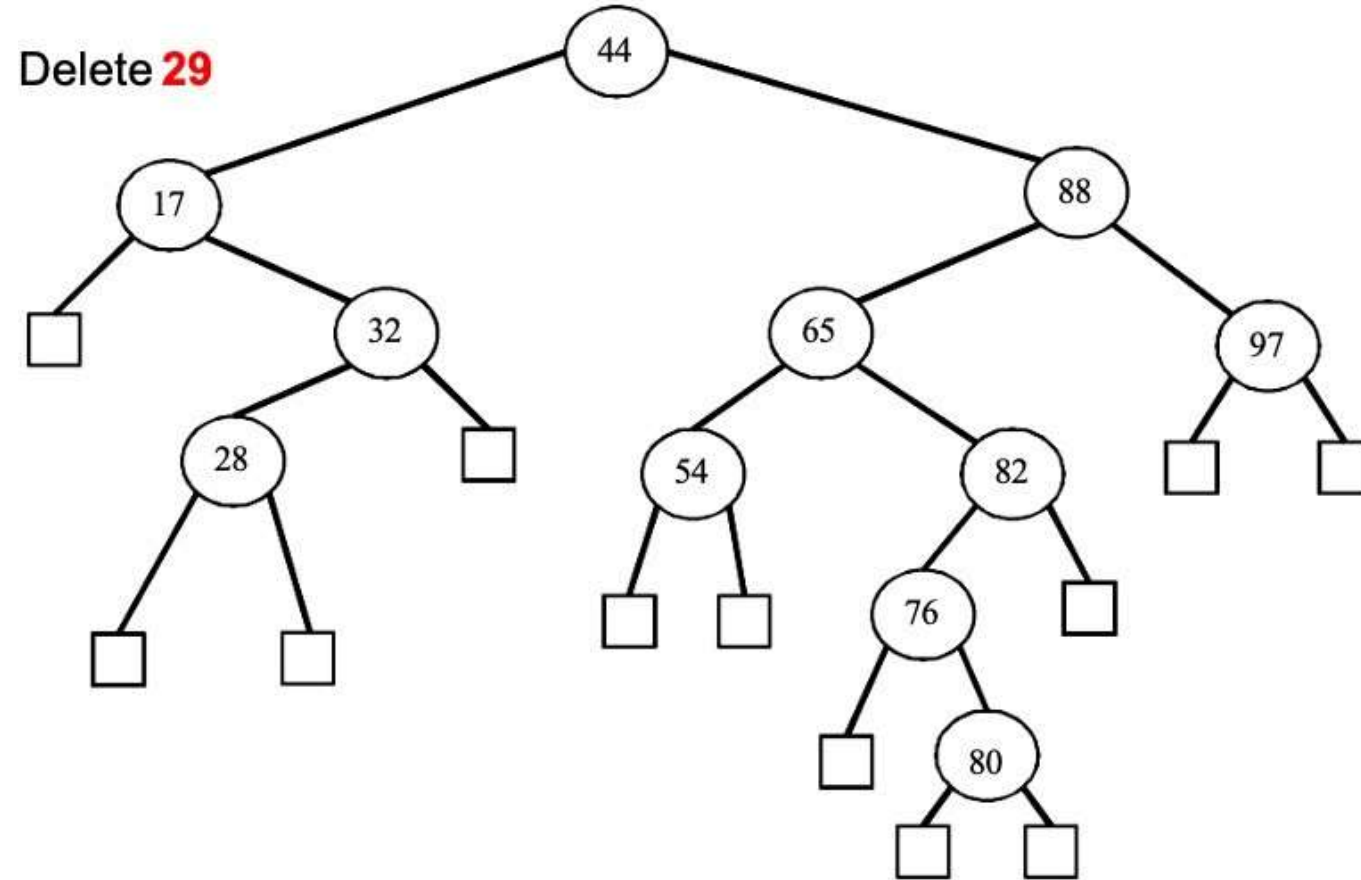
Operation on BST: Delete



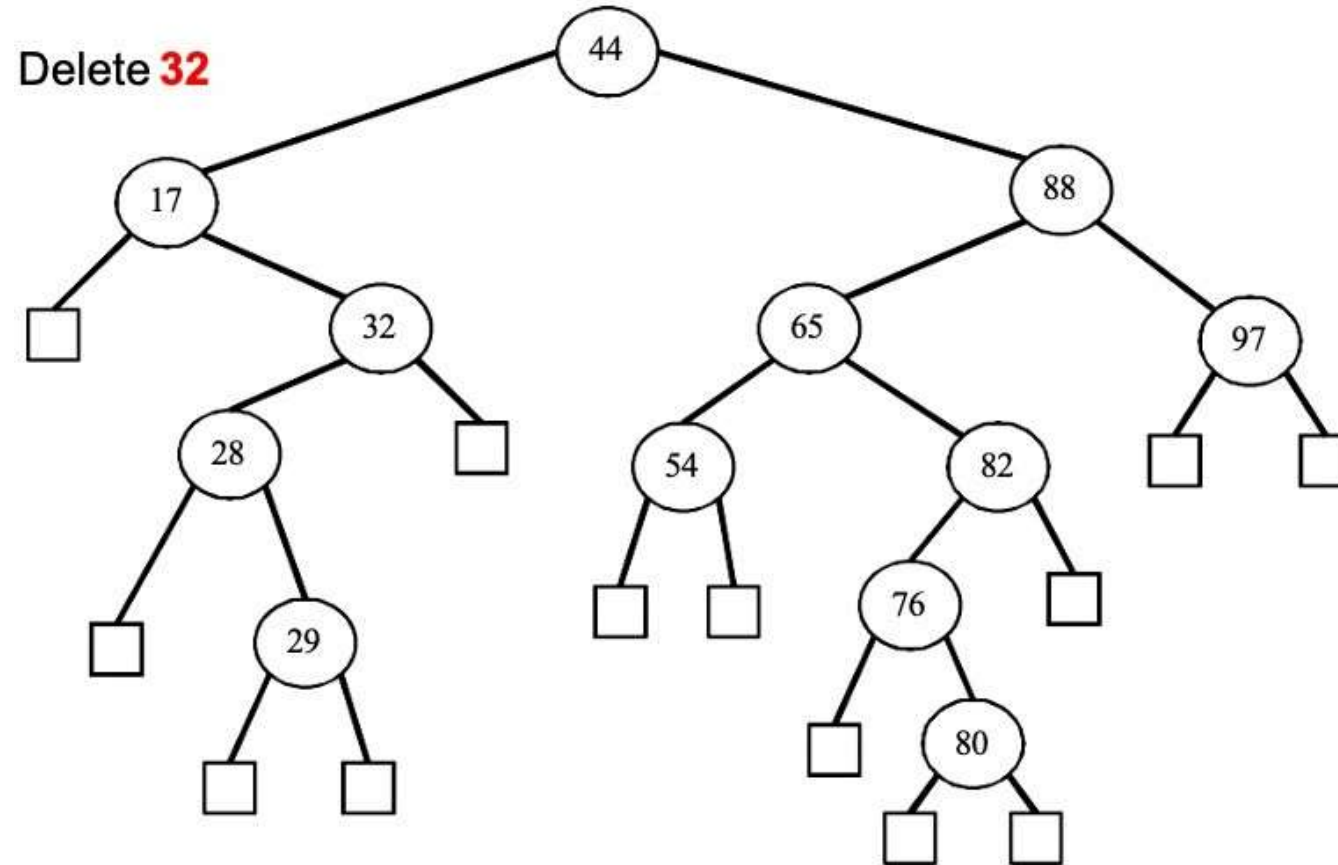
Operation on BST: Delete



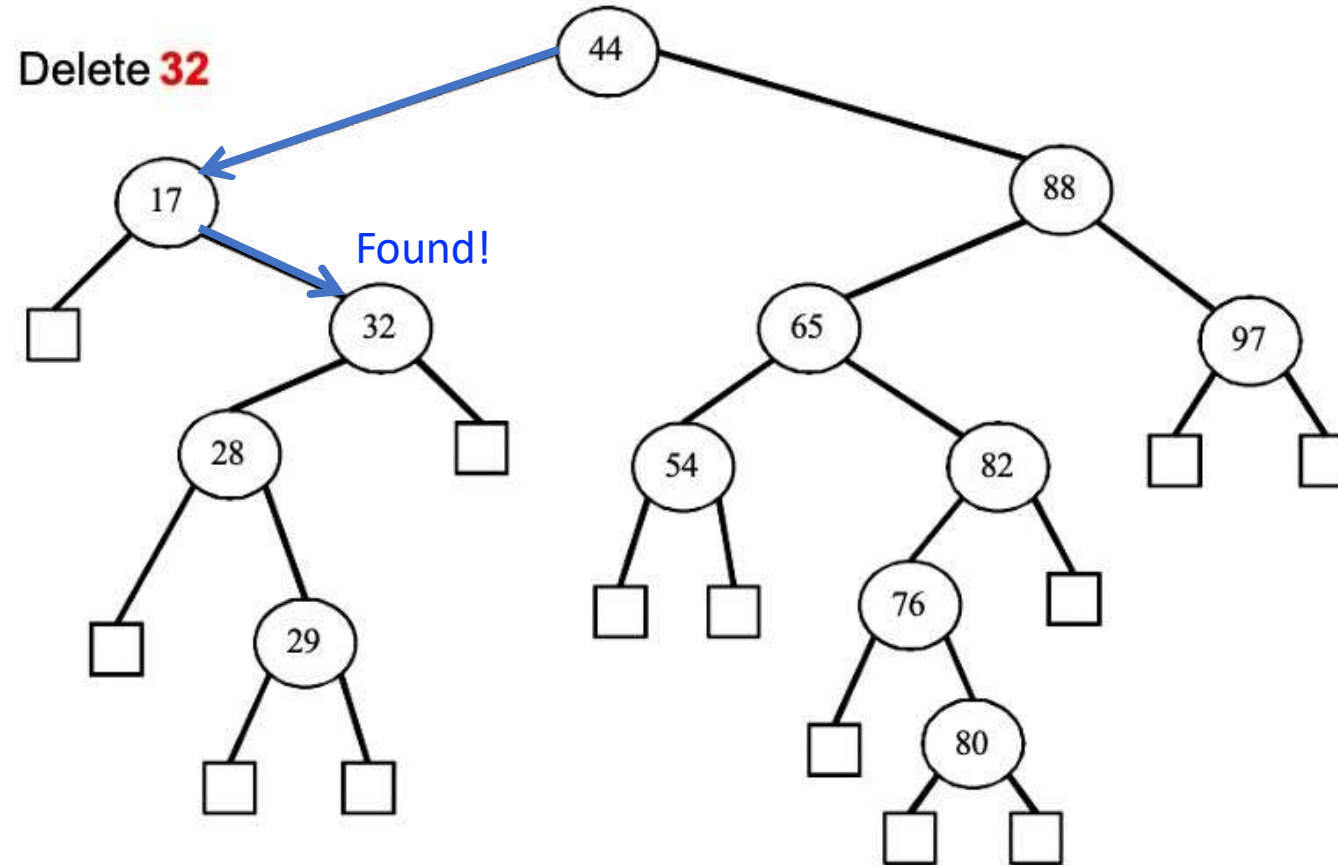
Operation on BST: Delete



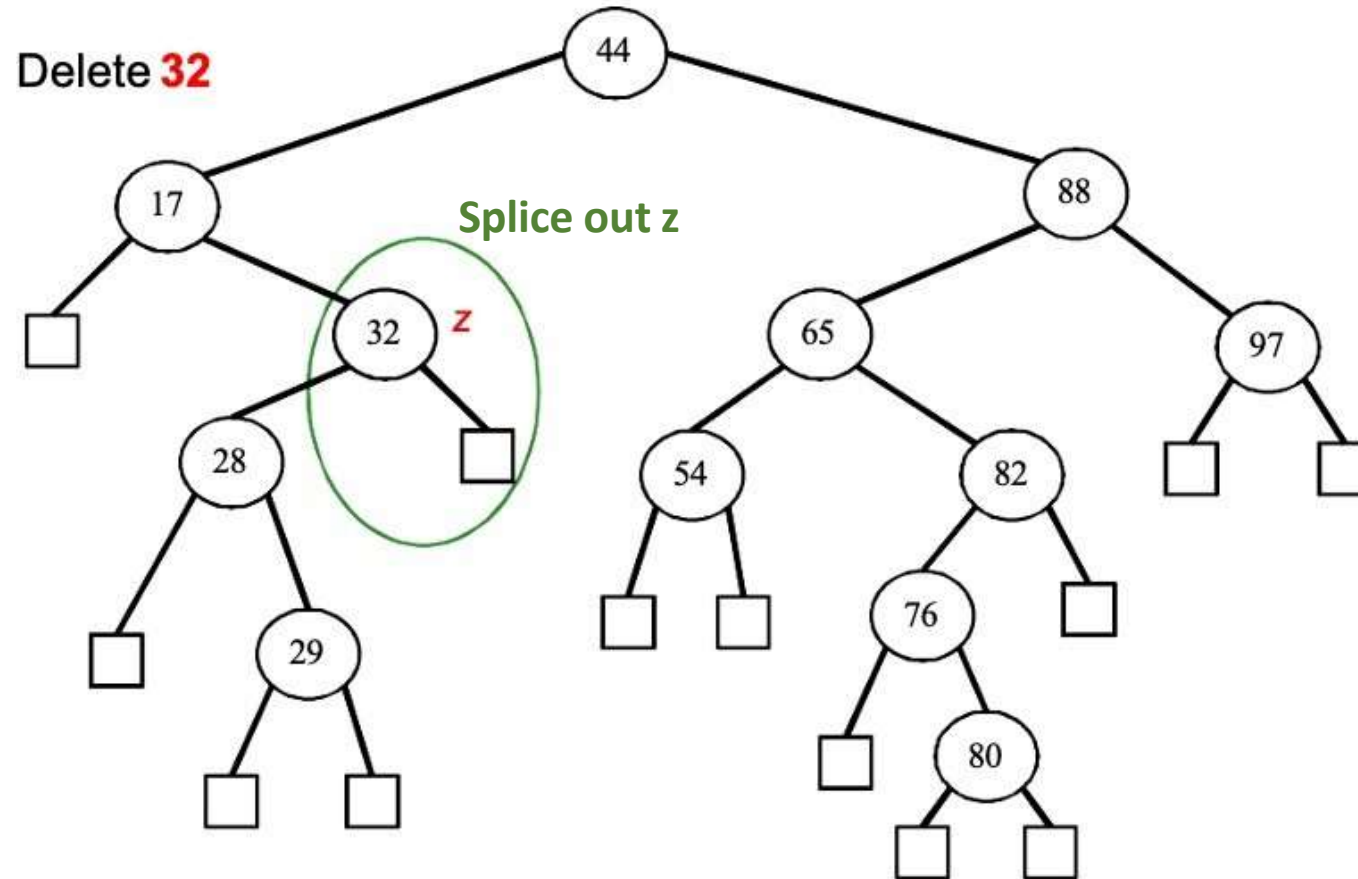
Operation on BST: Delete



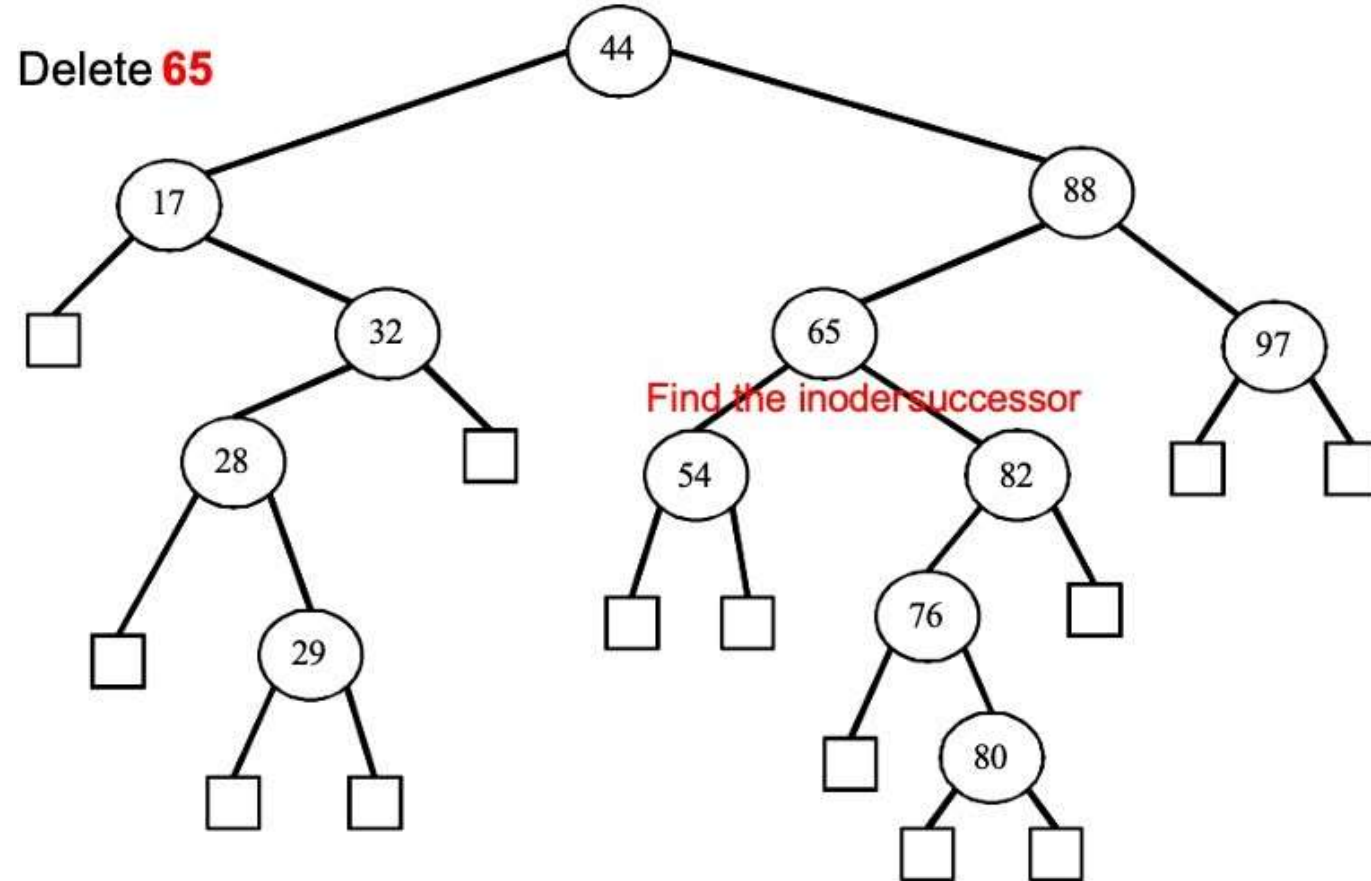
Operation on BST: Delete



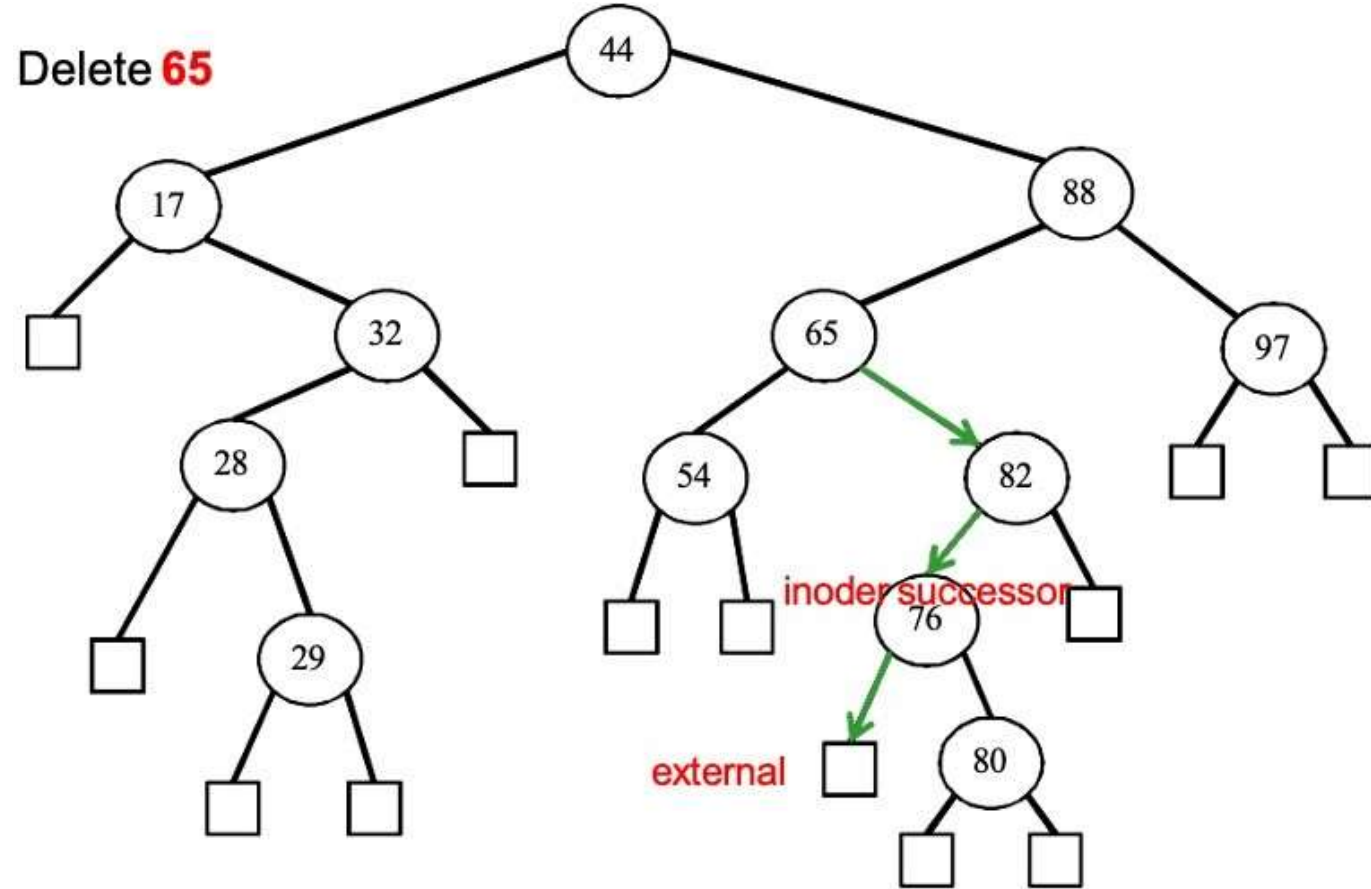
Operation on BST: Delete



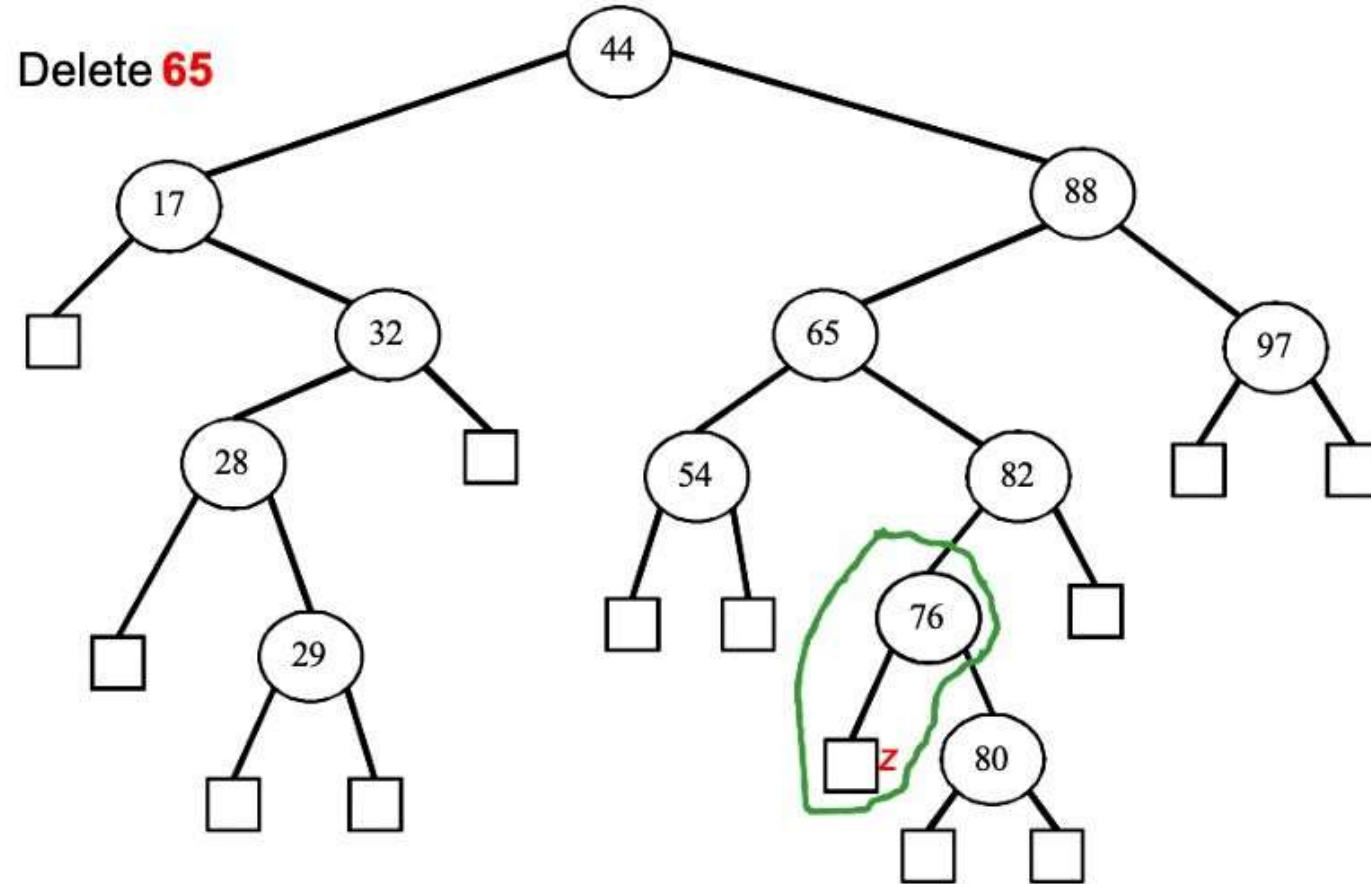
Operation on BST: Delete



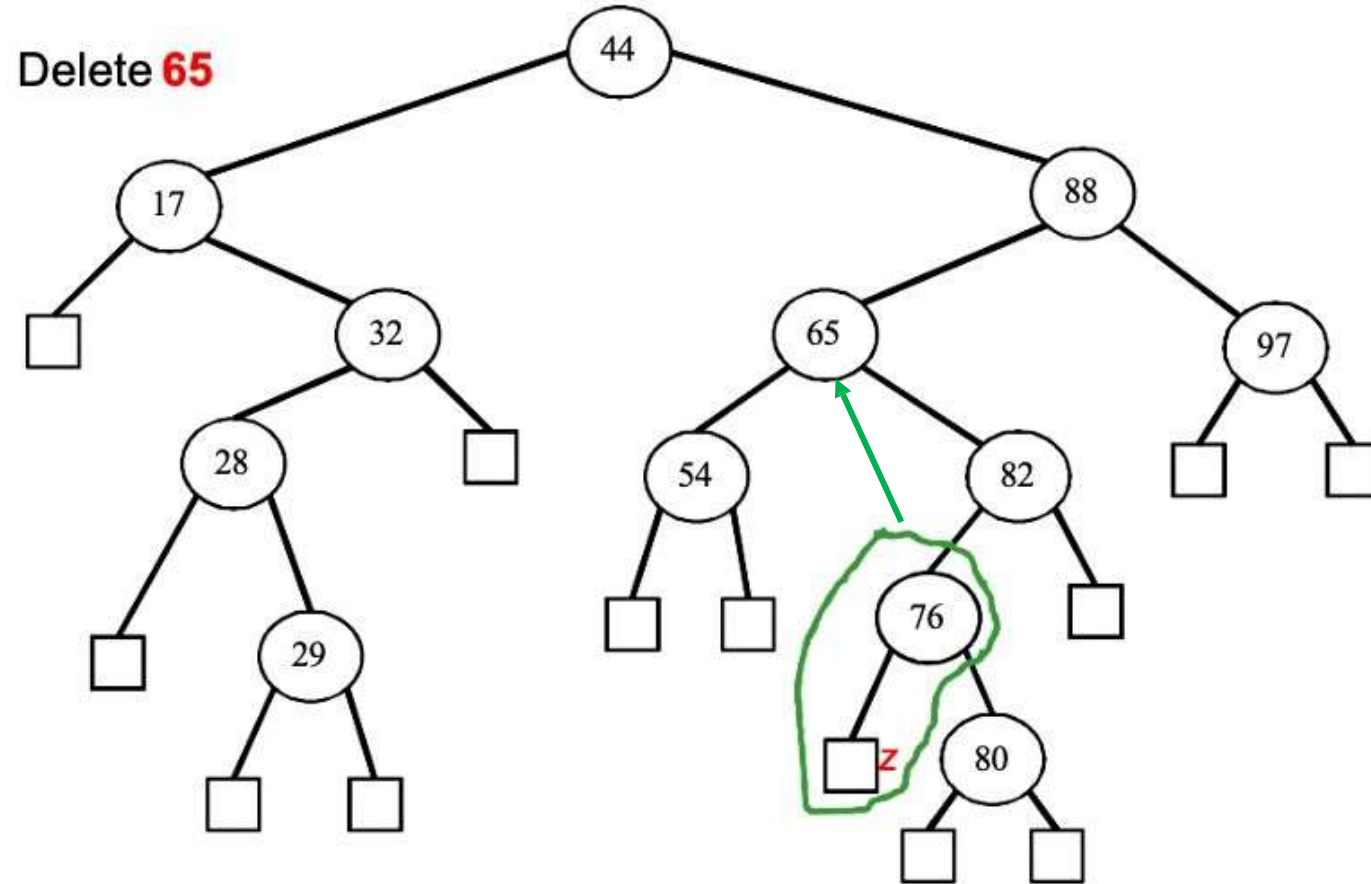
Operation on BST: Delete



Operation on BST: Delete



Operation on BST: Delete



Operation on BST: Simulation

- Draw the BST after doing each step of the following statements:
 - Insert 85
 - Delete 28
 - Delete 76
 - Insert 55
 - Delete 44

Sorting with BST

- Informal code for sorting array A of length n :

```
BSTSort(A)
  for i=1 to n
    TreeInsert(A[i]);
  InorderTreeWalk(root);
```

- *Running time is $O(n \lg n)$*
- *What will be the running time in the*
 - *Worst case?*
 - *Average case?*

BST: Remarks

- All complexity depends on height h
- $h = O(\lg n)$
- To guarantee performance:
 - Balanced tree !
- Randomly build tree
 - **Theorem**: The expected height of a randomly built binary search tree on n distinct keys is $O(\lg n)$

Balanced BST

- Keep $O(\lg n)$ height under dynamic operations
- General framework:
 - First a binary search tree
 - Maintain properties that ensure height guarantee
- AVL tree, red-black tree, ...

Codes for BST

```
#include<stdio.h> #include<stdlib.h>

typedef struct BSTnode{
    int key;
    struct BSTnode *left;
    struct BSTnode *right;
} BSTnode;

BSTnode *create( ){
    BSTnode *temp;
    printf("\nEnter key: ");
    temp = (BSTnode*)malloc(sizeof(BSTnode));
    scanf("%d", &temp->key);

    temp->left = temp->right = NULL;

    return temp;
}
```

Codes for BST

```
void insert(BSTnode *root, BSTnode *temp){
    if(temp->key < root->key){
        if(root->left != NULL)
            insert(root->left, temp);
        else
            root->left = temp;
    }
    if(temp->key > root->key){
        if(root->right != NULL)
            insert(root->right, temp);
        else
            root->right = temp;
    }
}
```

```
BSTnode *minValueNode(BSTnode *node){
    BSTnode *current = node;
    while (current->left!=NULL)
        current = current->left;
    return current;
}
```

Codes for BST

```
BSTnode* deleteNode(BSTnode *root, int key){
    if (root == NULL) return root;
    // If the key to be deleted is smaller than the root's key, then it lies in left subtree
    if (key < root->key)
        root->left = deleteNode(root->left, key);
    // If the key to be deleted is greater than the root's key, then it lies in right subtree
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
    // if key is same as root's key, then This is the node to be deleted
    else {
        // node with only one child or no child
        if (root->left == NULL) {
            BSTnode *temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            BSTnode *temp = root->left;
            free(root);
            return temp;
        }
    }
}
```

Codes for BST

```
// node with two children: Get the inorder successor (smallest in the right subtree)
BSTnode *temp = minValueNode(root->right);

// Copy the inorder successor's content to this node
root->key = temp->key;

// Delete the inorder successor
root->right = deleteNode(root->right, temp->key);
}

return root;
}

void inorder(BSTnode *root){
    if(root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}
```

Codes for BST

```
int main() {  
    char ch;  
    int item;  
    BSTnode *root = NULL, *temp;  
    do {  
        temp = create();  
        if(root == NULL)  
            root = temp;  
        else  
            insert(root, temp);  
        printf("\n Do you want to enter more(y/n)? ");   getchar();   scanf("%c", &ch);  
    } while(ch=='y'|ch=='Y');  
    printf("\n Inorder traversal of the tree:\t");  
    inorder(root);  
    do {  
        if(root == NULL){  
            printf("\n No key to delete!"); return 0; }  
        else {  
            printf("\nDelete what? ");   scanf("%d", &item);  
            root = deleteNode(root, item);  
            printf("\nInorder traversal of the modified tree \t"); inorder(root);  
        }  
        printf("\nDo you want to delete more(y/n)? ");   getchar();   scanf("%c", &ch);  
    } while(ch=='y'|ch=='Y'); return 0; }  
return 0; }
```


Thank You